SUMMARY: EXCEPTION HANDLING

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand the recursion function
- Understand the lambda function
- Understand what exception handling are
- Understand what the try statement is
- Understand what the try-except statement is
- Understand what the try-except-else statement is
- Understand what the try-except-finally statement is
- Understand what raise keyword is

KEY POINTS AND EXAMPLES:

Understand the recursion function

Recursion is a programming concept where a function calls itself to solve a problem. It is a way of breaking down a complex problem into smaller sub-problems of the same kind, until a base case is reached. Recursion differs from regular functions in several ways:

In recursion, a function calls itself with a slightly different input to accomplish the overall task. This self-reference is the defining characteristic of recursion.

Every recursive function must have a base case, which is a condition that stops the recursion and returns a value without calling the function again. Without a base case, the function would continue to call itself indefinitely, leading to an infinite loop and eventually causing a stack overflow error.

The recursive case is the part of the function where it calls itself with a different input. This input should be closer to the base case, so that the recursion eventually terminates.

Recursion function example:

```
def factorial(n):
    # Base case: If n is 0 or 1, return 1
    if n == 0 or n == 1:
        return 1
```

```
    # Recursive case: Calculate the factorial by calling the
function again
    # with a smaller value of n
    else:
        return n * factorial(n - 1)

print(factorial(5))  # Output: 120
```

In this example, the factorial function calls itself with n - 1 until the base case (n == 0 or n == 1) is reached. The recursive call factorial(n - 1) represents the sub-problem of calculating the factorial of n - 1, which is then multiplied by n to get the final result.

1. factorial(5) is called.
2. Since 5 is not 0 or 1, the recursive case is executed: return 5 * factorial(4).
3. factorial(4) is called, and the recursive case is executed again: return 4 * factorial(3).
4. factorial(3) is called, and the recursive case is executed again: return 3 * factorial(2).
5. factorial(2) is called, and the recursive case is executed again: return 2 * factorial(1).
6. factorial(1) is called, and the base case is reached: return 1.
7. The recursive calls start unwinding, and the calculations are performed: 1 * 2 * 3 * 4 * 5 = 120.

Regular function example:

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

a = factorial(5)
print(a) # Output: 120
```

In this regular function, we initialize result to 1, and then use a for loop to iterate from 1 to n. In each iteration, we multiply result by the current value of i. Finally, we return the accumulated result, which is the factorial of n.

1. result is initialized to 1.
2. The loop iterates from 1 to 5:

- Iteration 1: result = 1 * 1 = 1
- Iteration 2: result = 1 * 2 = 2

- Iteration 3: result = 2 * 3 = 6
- Iteration 4: result = 6 * 4 = 24
- Iteration 5: result = 24 * 5 = 120

3. The final value of result, which is 120, is returned.

Understand the lambda function

Lambda functions, also known as anonymous functions, are a concise way of defining small, one-line functions that can take any number of arguments but can only have a single expression in Python without needing to formally define them using the def keyword.

lambda is the keyword used to define a lambda function.

Syntax:

```
lambda arguments: expression
```

- arguments are the input parameters of the function, similar to the parameters you would define in a regular function.
- expression is the single expression or operation that the function evaluates and returns.

Note: Lambda functions cannot contain multiple expressions or statements, unlike regular functions.

```
add = lambda x, y: x + y
print(add(3, 5))
```

The output will be:

```
8
```

```
square = lambda x: x ** 2
result = square(5)
print(result)
```

The output will be:

```
25
```

We can combine regular functions with lambda functions.

```
def myfunc(n):
  return lambda a : a * n

mytripler = myfunc(3)

print(mytripler(11))
```

The output will be:

```
33
```

1. The myfunc function takes an argument n and returns a lambda function that takes another argument a.
2. Inside the lambda function, a is multiplied by n.
3. When myfunc(3) is called, it returns a lambda function where n is 3. This lambda function effectively becomes lambda a: a * 3.
4. mytripler now refers to this lambda function.
5. When mytripler(11) is called, it's equivalent to (lambda a: a * 3)(11), which results in 11 * 3, giving us the output 33.

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```

The output will be:

```
22
33
```

In this example, myfunc returns a lambda function that multiplies its argument by the value of n.

1. When myfunc(2) is called, it returns a lambda function where n is 2. This lambda function effectively becomes lambda a: a * 2.

2. mydoubler now refers to this lambda function.
3. Similarly, when myfunc(3) is called, it returns a lambda function where n is 3, effectively lambda a: a * 3, and mytripler refers to this lambda function.
4. When mydoubler(11) is called, it's equivalent to (lambda a: a * 2)(11), resulting in 11 * 2, which gives us the output 22.
5. Similarly, mytripler(11) is equivalent to (lambda a: a * 3)(11), resulting in 11 * 3, giving us the output 33.

Understand what exception handling are

Exceptions in Python are unexpected events or errors that occur during the execution of a program. When an error occurs, or exception as we call it, Python will normally stop and generate an error message.

Common definitions of errors encountered in Python programming:

1. **SyntaxError:** Occurs when the Python interpreter encounters an error in the syntax of the code, preventing it from executing. This can include issues like incorrect indentation, missing parentheses, or invalid keywords.
2. **IndentationError:** A specific type of SyntaxError that occurs when the indentation of code blocks is not consistent. Python uses indentation to define blocks of code, so incorrect or inconsistent indentation can lead to errors.
3. **TypeError**: Occurs when an operation or function is applied to an object of inappropriate type. For example, performing arithmetic on non-numeric data types or passing arguments of the wrong type to a function.
4. **NameError:** Occurs when a variable or function name used in the code is not defined or cannot be found in the current scope.
5. **ValueError:** Occurs when a function receives an argument of the right type but inappropriate value. For example, passing an invalid argument to a function that expects a certain range of values.
6. **IndexError:** Occurs when trying to access an element from a list, tuple, or string using an index that is out of its range. It indicates that the index provided does not exist in the sequence.
7. **KeyError:** Occurs when trying to access a dictionary with a key that does not exist in the dictionary. It indicates that the specified key is not found in the dictionary.
8. **AttributeError:** Occurs when trying to access an attribute or method that does not exist for a given object. It indicates that the object does not have the specified attribute or method.
9. **ZeroDivisionError:** Occurs when attempting to divide a number by zero, which is mathematically undefined. It indicates that the denominator in a division operation is zero.

Exception handling in programming is like having a safety net or contingency plan when

things go wrong. It's a way to gracefully handle unexpected situations or errors that may occur during the execution of your code, preventing it from crashing or behaving unexpectedly.

The four techniques or constructs for exception handling in Python are:

1. try
2. except
3. else
4. finally

Imagine you're a construction worker building a house. You follow the blueprint and instructions carefully, but sometimes things don't go as planned. For example, you might run out of materials, encounter a structural issue, or face unexpected weather conditions that disrupt your work.

Without proper contingency plans, any unexpected event could bring the entire construction process to a halt, leaving you with an incomplete and potentially unsafe structure. This is similar to a program crashing or behaving unpredictably when an error occurs.

Exception handling in programming is like having a contingency plan for these unexpected events. It's like having a backup plan or a set of instructions to follow when something goes wrong during the construction process.

1. **Try Block:** This is like the normal construction process where you're following the blueprint and instructions. You're doing the work, and everything is going smoothly.
2. **Exceptions:** These are the unexpected events or errors that can occur during construction, like running out of materials, encountering a structural issue, or facing bad weather conditions.
3. **Except Blocks:** These are your contingency plans or backup strategies for dealing with specific types of unexpected events or errors. Just like you might have a plan for what to do if you run out of materials or if the weather turns bad, your except blocks handle specific types of exceptions or errors that may occur in your code.
4. **Else Block:** This is like the final inspection or sign-off when the construction is completed without any issues. If no exceptions occurred during the construction process, you can proceed with the next steps or celebrate a successful completion.
5. **Finally Block:** This is like the cleanup or tear-down phase, where you ensure that all resources are properly handled or released, regardless of whether the construction was successful or not. For example, you might need to clean up the construction site or return any rented equipment, even if the project was interrupted by an unexpected event.

Understand what try statement is

The try statement is used to enclose a block of code that might raise an exception during execution. It allows you to separate the code that might cause an exception from the code that handles the exception. However, it is often used in combination with the except, else, and finally blocks to provide a comprehensive and robust exception handling strategy.

Syntax:

```
try:
    # Code that might raise an exception
    # ...
```

- When the try block is encountered, Python starts executing the code within it.
- If no exceptions are raised during the execution of the code inside the try block, the code runs normally until the end of the try block.
- If an exception occurs within the try block, Python immediately stops executing the remaining code in the try block and looks for an appropriate except block to handle the exception.

Usage without except Block:
While it is possible to use the try statement without an except block, it is generally not recommended because it defeats the purpose of exception handling. If an exception is raised within the try block and there is no except block to catch it, the exception will propagate up the call stack until it is caught by an enclosing try-except block or reaches the top level of the program, resulting in an unhandled exception error and termination of the program.

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
```

In this example:

- The try block attempts to read user input, convert it to an integer, and perform a division operation.
- If the user enters a non-numeric value, a ValueError will be raised.
- If the user enters 0, a ZeroDivisionError will be raised.
- Since there are no except blocks to catch and handle these exceptions, the program will terminate with an unhandled exception error if either of these exceptions occurs.

Understand what the try-except statement is

The except block(s) are used to catch and handle specific exceptions raised within the try block. You can have multiple except blocks to handle different types of exceptions.

Syntax:

```
try:
    # Code that might raise an exception
    # ...
except Exception1:
    # Code to handle Exception1
    # ...
except Exception2:
    # Code to handle Exception2
    # ...
```

When an exception occurs within the try block, Python immediately stops executing the remaining code in the try block and looks for an appropriate except block to handle the exception. The except blocks are checked in the order they appear, and the first matching exception type is handled.

If a matching except block is found, the corresponding exception handler code is executed, and the program continues execution after the entire try-except construct. If no matching except block is found, the exception is propagated up the call stack until it is caught by an enclosing try-except block or reaches the top level of the program, resulting in an unhandled exception error.

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
except ValueError:
    print("Error: Invalid input. Please enter a number.")
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
```

The try block attempts to read user input, convert it to an integer, and perform a division operation.
If the user enters a non-numeric value, a ValueError is raised, and the corresponding except block handles the error by printing an appropriate message.
If the user enters 0, a ZeroDivisionError is raised, and the corresponding except block handles the error by printing an appropriate message.
If any other exception occurs, it is not caught by the provided except blocks and will result in an unhandled exception error.

If you enter a non-numeric value, for example, abc, the output will be:

```
Enter a number: abc
Error: Invalid input. Please enter a number.
```

If we don't provide the appropriate exception type in the except block, Python will not be able to handle the specific exception correctly. Instead, the exception will propagate up the call stack until it is caught by a more general except block (if any) or reaches the top level of the program, resulting in an unhandled exception error.

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
except ValueError:
    print("Error: Invalid input. Please enter a number.")
```

In this case, if you enter 0 as input, a ZeroDivisionError will be raised due to the division by zero operation (10 / 0). However, the provided except block only catches ValueError exceptions and does not handle ZeroDivisionError.

The output in this case will be:

```
Enter a number: 0
Traceback (most recent call last):
  File "<path/to/file.py>", line 3, in <module>
    result = 10 / x
ZeroDivisionError: division by zero
```

Since there is no except block to handle the ZeroDivisionError, Python prints a traceback showing the unhandled exception, including the location where the exception occurred (result = 10 / x), the type of exception (ZeroDivisionError), and the exception message (division by zero).

To handle this situation correctly, we should either:

1. Add an except block specifically for ZeroDivisionError:

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
```

```
except ValueError:
    print("Error: Invalid input. Please enter a number.")
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
```

2. Use a bare except block to catch all exceptions (not recommended, as it can catch unintended exceptions):

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print(f"Result: {result}")
except ValueError:
    print("Error: Invalid input. Please enter a number.")
except:
    print("An error occurred during the calculation.")
```

```
try:
  print(x)
except:
  print("An exception occurred")
```

The output will be:

```
An exception occurred
```

- The code enters the try block and attempts to print the value of the variable x.
- Since x is not defined or initialized before the print(x) statement, a NameError exception is raised. The NameError occurs because Python cannot find the variable x in the current scope.
- The except block catches the NameError exception (and any other exception that might occur in the try block).
- Since there is an exception raised in the try block, and a matching except block is found, the code inside the except block is executed, printing "An exception occurred".
- After executing the except block, the try-except construct is completed, and the program continues with any remaining code after the try-except block (if there is any).

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
```

```
except:
    print("Something else went wrong")
```

The output will be:

```
Variable x is not defined
```

- The code enters the try block and attempts to print the value of the variable x.
- Since x is not defined or initialized before the print(x) statement, a NameError exception is raised.
- Python checks the except blocks in the order they appear.
- Since a NameError was raised in the try block, and the first except block matches this exception type, the code inside that block is executed, printing "Variable x is not defined".
- After executing the appropriate except block, the try-except construct is completed, and the program continues with any remaining code after the try-except block (if there is any).

In Python, we can use multiple exceptions in a single except block to handle different types of exceptions in a similar way. This can be useful when you want to execute the same code for multiple types of exceptions.

Syntax:

```
try:
    # Code that may raise exceptions
    ...
except (ExceptionType1, ExceptionType2, ...):
    # Handle ExceptionType1 or ExceptionType2 or ...
    # Common handling code
    ...
```

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except (ValueError, ZeroDivisionError):
    print("Invalid input or division by zero")
```

In this example:

- If the user enters a non-integer input (ValueError), or
- If the user enters 0 causing division by zero (ZeroDivisionError),the except block will

handle both exceptions by printing "Invalid input or division by zero".

Using nested try-except blocks to handle both ValueError and ZeroDivisionError exceptions appropriately.

```
try:
    x = int(input("Enter a number: "))
    try:
        result = 10 / x
        print(f"Result: {result}")
    except ZeroDivisionError:
        print("Division by zero inside nested try block.")
except ValueError:
    print("Invalid input. Please enter a number.")
```

In this example:

- The outer try block attempts to execute the code that prompts the user to input a number
- Inside the outer try block, there is a nested try-except block:
    - The nested try block attempts to perform a division (result = 10 / x).
    - If ZeroDivisionError occurs (i.e., if the user inputs 0), it will catch the exception and print "Division by zero inside nested try block.".
- If the conversion of user input to an integer (int(input("Enter a number: "))) raises a ValueError, the outer except block will catch it and print "Invalid input. Please enter a number.".

Understand what the try-except-else statement is

The else block in Python's exception handling provides a way to define a block of code that should only run if no exceptions occur within the corresponding try block.The else block in exception handling is optional. It's not mandatory to include an else block in a try-except statement. The presence of the else block depends on whether there's additional code that should only run if no exceptions occur within the corresponding try block.

Its purpose is to separate the normal code execution path from the exception handling code, improving code readability and organization.

The else block is placed immediately after the except block(s) in a try-except statement.

Syntax:

```
try:
```

```
    # Code that might raise an exception
    # ...
except Exception1:
    # Code to handle Exception1
    # ...
except Exception2:
    # Code to handle Exception2
    # ...
else:
    # Code to execute if no exceptions were raised
    # ...
```

With else block:

```
# Example with else block
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error: Division by zero!")
else:
    print("Division successful. Result:", result)
```

The output will be:

```
Division successful. Result: 5.0
```

In this example, the else block contains code that prints the result of the division if no exception occurs. If there's a division by zero error, the except block will handle it, and the else block will be skipped.

Without else bock:

```
# Example without else block
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Division by zero!")
```

The output will be:

```
Error: Division by zero!
```

In this example, there's no else block. If a division by zero error occurs, the except block handles it by printing an error message. Since there's no additional code to execute when no exceptions occur, there's no need for an else block.

```
try:
  print("Hello")
except:
  print("Something went wrong")
else:
  print("Nothing went wrong")
```

The output will be:

```
Hello
Nothing went wrong
```

- The try block contains the code print("Hello"), which is expected to execute without any exceptions.
- Since there are no errors in the try block, the except block is not executed.
- As a result, the else block is executed, printing "Nothing went wrong" to the console.

```
my_list = [1, 2, 3]

try:
    print(my_list[4])  # Accessing an index that doesn't exist
except IndexError:
    print("IndexError: Index out of range")
else:
    print("No IndexError occurred")
```

The output will be:

```
IndexError: Index out of range
```

- The try block attempts to access an index (my_list[4]) that doesn't exist in the list my_list, causing an IndexError.
- The except block catches the IndexError and prints a corresponding error message.
- Since there's an exception, the else block is skipped, and its code is not executed.

```
my_dict = {"name": "Alice", "age": 30}
```

```
try:
    my_dict.add("city", "New York")  # Attempting to use the
non-existent add() method
except AttributeError:
    print("AttributeError: 'dict' object has no attribute 'add'")
else:
    print("No AttributeError occurred")
```

The output will be:

```
AttributeError: 'dict' object has no attribute 'add'
```

- The try block attempts to use the add() method on the dictionary my_dict.
- However, dictionaries in Python do not have an add() method, so trying to call add() raises an AttributeError.
- The except block catches the AttributeError and prints a corresponding error message.
- Since there's an exception, the else block is skipped, and its code is not executed.

```
my_dict = {"name": "Alice", "age": 30}

try:
    print(my_dict["email"])  # Accessing a key that doesn't exist
in the dictionary
except KeyError:
    print("KeyError: Key not found")
else:
    print("No KeyError occurred")
```

The output will be:

```
KeyError: 'email'
```

- The try block attempts to access a key ("email") in the dictionary my_dict that doesn't exist, causing a KeyError.
- The except block catches the KeyError and prints a corresponding error message.
- Since there's an exception, the else block is skipped, and its code is not executed.

```
try:
    result = 10 / "2"  # Performing division with a string
```

```
except TypeError:
    print("TypeError: Unsupported operand types")
else:
    print("No TypeError occurred")
```

The output will be:

```
TypeError: Unsupported operand types
```

- The try block attempts to perform a division operation (10 / "2") where one of the operands is a string, which is not supported and raises a TypeError.
- The except block catches the TypeError and prints a corresponding error message.
- Since there's an exception, the else block is skipped, and its code is not executed.

Understand what the try-except-finally statement is

The finally block in Python's exception handling is a block of code that is always executed, regardless of whether an exception occurs within the corresponding try block or not.The finally block is optional in Python's exception handling. It's not mandatory to include a finally block in a try-except statement.

Syntax:

```
try:
    # Code that might raise an exception
    # ...
except Exception1:
    # Code to handle Exception1
    # ...
except Exception2:
    # Code to handle Exception2
    # ...
finally:
    # Code to execute regardless of exceptions
    # ...
```

The primary purpose of the finally block is to ensure that certain cleanup operations or resource release actions are performed, even if an exception occurs or the program flow is interrupted.

```
try:
    user_input = int(input("Enter a number: "))
    result = 10 / user_input
    print(f"Result: {result}")
except ValueError:
    print("Error: Invalid input. Please enter a number.")
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
finally:
    print("Thank you for using this program.")
```

The output will be:

```
Error: Division by zero is not allowed.
Thank you for using this program.
```

- The code enters the try block and prompts the user to enter a number using the input function.
- If the user enters a non-numeric value, a ValueError is raised, and the corresponding except block handles the error by printing the message "Error: Invalid input. Please enter a number."
- If the user enters 0, a ZeroDivisionError is raised, and the corresponding except block handles the error by printing the message "Error: Division by zero is not allowed."
- If the user enters a valid numeric value other than 0, the division operation is performed (10 / user_input), and the result is printed using the print(f"Result: {result}") statement.
- Regardless of whether an exception was raised or not, the finally block is executed, and the message "Thank you for using this program." is printed.

```
def divide_numbers(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed.")
    else:
        print(f"Result: {result}")
    finally:
        print("Operation completed.")

divide_numbers(10, 2)  # Both else and finally blocks will execute
divide_numbers(5, 0)   # Only finally block will execute
```

The output will be:

```
Result: 5.0
Operation completed.
Error: Division by zero is not allowed.
Operation completed.
```

1. When divide_numbers(10, 2) is called:
   - The try block successfully divides 10 by 2, and the result is stored in the result variable.
   - Since no exceptions were raised in the try block, the else block is executed, printing "Result: 5.0".
   - Finally, the finally block is executed, printing "Operation completed.".

2. When divide_numbers(5, 0) is called:
   - The try block attempts to divide 5 by 0, which raises a ZeroDivisionError.
   - The except block catches the ZeroDivisionError and prints "Error: Division by zero is not allowed.".
   - Since an exception was raised in the try block, the else block is skipped.
   - However, the finally block is still executed, printing "Operation completed.".

Difference between the else block and finally block:

Aspect	**else Block**	**finally Block**
Purpose	To specify code that should run when the try block completes successfully without raising any exceptions.	To specify cleanup operations or release resources, regardless of whether an exception was raised or not.
Execution Condition	Executed only if no exceptions were raised in the try block.	Always executed, regardless of whether an exception was raised or not.
Skipped When	Skipped if an exception is raised and caught by an except block.	Never skipped.
Execution Order	Executed after the try block (if no exceptions) and before the finally block.	Executed after the try block and any except or else blocks.
Optional or Required	Optional	Optional

Understand what raise keyword is

The raise keyword is used to generate an exception intentionally. It allows you, as the programmer, to create and "raise" (or "throw") an exception at a specific point in your code.

Syntax:

```
raise ExceptionType("Error message")
```

We can use raise to create and throw one of Python's built-in exceptions.

```
x = 10
if x > 5:
    raise ValueError("Value is too high")
```

The output will be:

```
 File "./prog.py", line 3, in <module>
ValueError: Value is too high
```

The condition x > 5 is true when x = 10, and that's why the ValueError exception is raised with the message "Value is too high".

If the condition in the if statement is false, then the code block under the if statement (including the raise statement) will not be executed.

```
x = 3

if x > 5:
    raise ValueError("Value is too high")

print("This will be printed.")
```

The output will be:

```
This will be printed.
```

In this case, the value of x is 3, which is not greater than 5. Therefore, the condition x > 5 is false. As a result, the code block under the if statement, which includes the raise ValueError("Value is too high") statement, will be skipped.

Instead, the program will execute the next line of code after the if block, which is the print("This will be printed.") statement.

No exception will be raised because the condition that triggers the raise statement is not met.

If you want to explicitly raise an exception when a certain condition is not met, you can use the else block along with the if statement

```
x = 3

if x > 5:
    print("Value is high.")
else:
    raise ValueError("Value is too low.")

print("This will not be printed.")
```

The output will be:

```
 File "./prog.py", line 6, in <module>
ValueError: Value is too low.
```

In this example, since the condition x > 5 is false, the else block will be executed, and a ValueError exception will be raised with the message "Value is too low."

```
def divide_numbers(a, b):
    try:
        if b == 0:
            raise ZeroDivisionError("Division by zero is not
allowed.")
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed")
        return None

print(divide_numbers(10, 2))
print(divide_numbers(10, 0))
```

The output will be:

```
5.0
Error: Division by zero is not allowed
None
```

1. When divide_numbers(10, 2) is called:
   - The function enters the try block.
   - The condition b == 0 is false, so the raise ZeroDivisionError statement is skipped.
   - The division operation a / b is performed, and the result 5.0 is returned.

2. When divide_numbers(10, 0) is called:
   - The function enters the try block.
   - The condition b == 0 is true, so a ZeroDivisionError exception is raised with the message "Division by zero is not allowed."
   - The except block catches the ZeroDivisionError exception.
   - The error message is printed using print(f"Error: {e}").
   - The function returns None.